



Departamento de Tecnología

Desarrollo 4D

Prueba Técnica

La presente prueba técnica consiste en el desarrollo de una aplicación web **Full Stack** que simula un sistema de gestión académica para la administración de estudiantes y cursos.

El objetivo principal es construir una solución completa que permita realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre las entidades principales del sistema, así como gestionar la asignación de cursos a estudiantes mediante una relación intermedia en la base de datos.

La segmentación también consta de dos roles básicos como lo son Administrador y estudiante donde el administrador puede asignar usuarios a cursos y el estudiante solamente puede ver sus cursos asignados

La aplicación deberá estar compuesta por:

- Un **backend desarrollado en Nest.js**, encargado de exponer una API RESTful segura mediante autenticación con JWT, manejar la lógica de negocio y administrar la persistencia de datos en una base de datos relacional (MySQL o SQL Server).
- Un **frontend desarrollado en Angular**, responsable de consumir la API, gestionar la autenticación del usuario y proporcionar una interfaz gráfica intuitiva para la administración de estudiantes, cursos y sus respectivas asignaciones.

Por favor, sigue al pie de la letra las indicaciones. **Ten en cuenta que si incumples alguno de los siguientes puntos, tu prueba será invalidada y serás descartado de forma inmediata:**

1. **No usar ChatGPT o alguna IA de ayuda o autocompletado** (se permite el uso de documentación oficial de la tecnología a utilizar).

Se te evaluará en los siguientes aspectos:

1. **Diseño de la aplicación:** debe ser **presentable y limpio**.
2. **Código:** Debes seguir buenas prácticas de la tecnología utilizada y emplear los principios **SOLID** y **CLEAN CODE**.
3. **Diseño de base de datos:** Debe de presentarse una base de datos bien organizada y debe de seguir las reglas de normalización.
4. **Buenas prácticas:** Seguir las buenas prácticas de la tecnología utilizada. Esto incluye la estructura del proyecto, nombrado de variables, modularización del código, y manejo de dependencias.

Backend

Objetivo: Crear una aplicación Node.js con funcionalidades CRUD (Crear, Leer, Actualizar, Eliminar) para manejar registros en una base de datos (MySQL o SQLServer) a través de una API RESTful, manejo de roles de estudiante y administrador

Requisitos de la Aplicación:

1. **CRUD Completo:** La aplicación debe permitir crear, leer, actualizar y eliminar registros en la base de datos.
2. **Manejo de Errores:** Implementar un manejo adecuado de errores y asegurar que la API responda con los códigos de estado HTTP apropiados.
3. **Inicio de sesión** (Sign-in/Sign-up)
3. **Diseño de la Base de Datos:** Diseñar una base de datos que contenga al menos tres tablas:
 - **Estudiante (T1):** Tabla que contiene información de los estudiantes.
 - **Curso (T2):** Tabla que contiene información de los cursos.
 - **Curso-Estudiante (T3):** Tabla intermedia que relaciona estudiantes con cursos.
4. **Datos a Obtener:**
 - Lista de cursos.
 - Lista de estudiantes.
 - Lista de estudiantes con sus cursos asignados.
5. **Consumo del Servicio:** Los endpoints de la API deben ser probados utilizando Postman.
6. **Middleware de Autenticación:** Implementar un middleware de autenticación básico utilizando Node.js y Express. El middleware debe verificar si el usuario ha iniciado sesión antes de permitir el acceso a una ruta protegida. Utilizar JWT (JSON Web Tokens) para manejar la autenticación.
7. Response deseado del backend:

1. CREAR:

```
{
  ...."idEstudiante": "<ID>",
  ...."nombre": "<Nombre del estudiante>",
  ...."apellido": "<Apellidos del estudiante>",
  ...."nivel": "<Nivel>", //p.e.: Diversificado
  ...."seccion": "<sección>"
}

{
  ...."idCurso": "<código del curso>",
  ...."nombreCurso": "<nombre del curso>",
  ...."idGrado": "<código grado>", //p.e.: 4TO
  ...."idCarrera": "<carrera>", //p.e.: Bachillerato
  ...."catedratico": [
    ....{
      ...."idCatedratico": "<código del catedrático>", //p.e.: OGARCIA
      ...."nombreCatedratico": "<Nombre del catedrático>"
    }
  ]
}
```

2. LEER:

```
{
  ...."idEstudiante": "<ID>",
  ...."nombre": "<Nombre del estudiante>",
  ...."apellido": "<Apellidos del estudiante>",
  ...."nivel": "<Nivel>", //p.e.: Diversificado
  ...."grado": "<Grado>", //p.e.: 4to
  ...."idCarrera": "<código de carrera>", //p.e.: Bachillerato
  ...."seccion": "<sección>",
  ...."cursosAsignados": [
    ....{
      ...."idCurso": "<código del curso>",
      ...."nombreCurso": "<nombre del curso>"
    }
  ]
}
```

3. ACTUALIZAR:

```
{
  ...."idEstudiante": "<ID>",
  ...."nombre": "<Nuevo nombre de estudiante>"
}
```

4. ELIMINAR:

```
{
  ...."idEstudiante": "<ID>"
}
```

Detalles Específicos:

- Tecnologías: Utilizar Nest.js para el desarrollo de la aplicación.
- Base de Datos: SQLServer.

- Endpoints RESTful: Asegurarse de seguir los principios REST en el diseño de los endpoints.
- Estructura de Respuesta: Asegurarse de que las respuestas de la API sigan un formato JSON estandarizado.

Entregables:

1. Código Fuente: Subir el código fuente a un repositorio en GitHub (o similar) y compartir el enlace.
2. Documentación: Incluir un archivo README.md en el repositorio con:
 - a. Instrucciones de instalación y ejecución de la aplicación.
 - b. Descripción de los endpoints de la API.
 - c. Ejemplos de uso de los endpoints (puede incluir capturas de pantalla de Postman).
 - d. Script de Base de Datos: Proveer scripts SQL para la creación y población de las tablas.

Frontend

Objetivo

Desarrollar una aplicación frontend utilizando Angular que consuma la API REST desarrollada en el backend (Node.js), permitiendo la gestión visual de los registros mediante una interfaz amigable y funcional.

Requisitos de la Aplicación

1. **Consumo de API REST:**

El frontend debe consumir los endpoints desarrollados en el backend mediante servicios HTTP, respetando los principios RESTful.

2. Módulos Funcionales:

La aplicación debe contar como mínimo con los siguientes módulos o vistas:

- Sign-in / Sign-up
- Gestión de estudiantes.
- Gestión de cursos.
- Asignación de cursos a estudiantes.
- Visualización de estudiantes con sus cursos asignados.

3. CRUD Completo:

Desde la interfaz gráfica el usuario debe poder:

- Crear registros.
- Visualizar listados.
- Actualizar información existente.
- Eliminar registros.

4. Autenticación:

- Implementar un formulario de inicio de sesión.
- Consumir el endpoint de autenticación del backend.
- Almacenar el token JWT recibido.
- Enviar el token en las solicitudes a rutas protegidas.
- Restringir el acceso a las vistas si el usuario no está autenticado.

5. Manejo de Errores:

La aplicación debe manejar errores provenientes del backend mostrando mensajes claros al usuario cuando ocurra algún fallo en las operaciones.

6. Diseño de la Interfaz:

- El diseño debe ser limpio, ordenado y funcional.
- Debe facilitar la navegación entre los distintos módulos.
- Se evaluará la claridad visual y la experiencia de usuario.

7. Buenas Prácticas:

- Uso correcto de componentes, servicios y módulos.
- Separación de responsabilidades.

- Uso adecuado del sistema de rutas de Angular.
- Protección de rutas mediante GUARD.
- Código legible y mantenible siguiendo principios SOLID y CLEAN CODE.

Entregables

1. Código fuente del frontend incluido en el repositorio.
2. Instrucciones en el README.md para la instalación y ejecución del frontend.
3. Evidencia visual del funcionamiento de la aplicación (capturas de pantalla opcionales).